Type 3 - Simulation (Performance) Study

Shahab Afshar (CprE 558)

CPR E 458/558 Real-Time Systems, Fall 2025

Department of Electrical and Computer Engineering

Iowa State University

Abstract

This project implements and evaluates server-based scheduling algorithms for mixed periodic-aperiodic real-time workloads. A comprehensive discrete-event simulator was developed in Python/Streamlit, implementing four server-based algorithms (Polling Server, Deferrable Server, Sporadic Server, and Background Scheduler) along with foundational scheduling algorithms (RMS, EDF, DMS, LLF). The simulator provides interactive Gantt chart visualizations, schedulability analysis, and performance metrics including CPU utilization, context switches, deadline misses, and aperiodic response times. Evaluation using deterministic benchmark scenarios demonstrates correct implementation of server capacity management policies: Polling Server loses unused capacity, Deferrable Server preserves capacity within periods, and Sporadic Server provides dynamic replenishment for optimal aperiodic response times. The simulator serves as both an educational tool for understanding real-time scheduling concepts and a platform for comparative algorithm analysis.

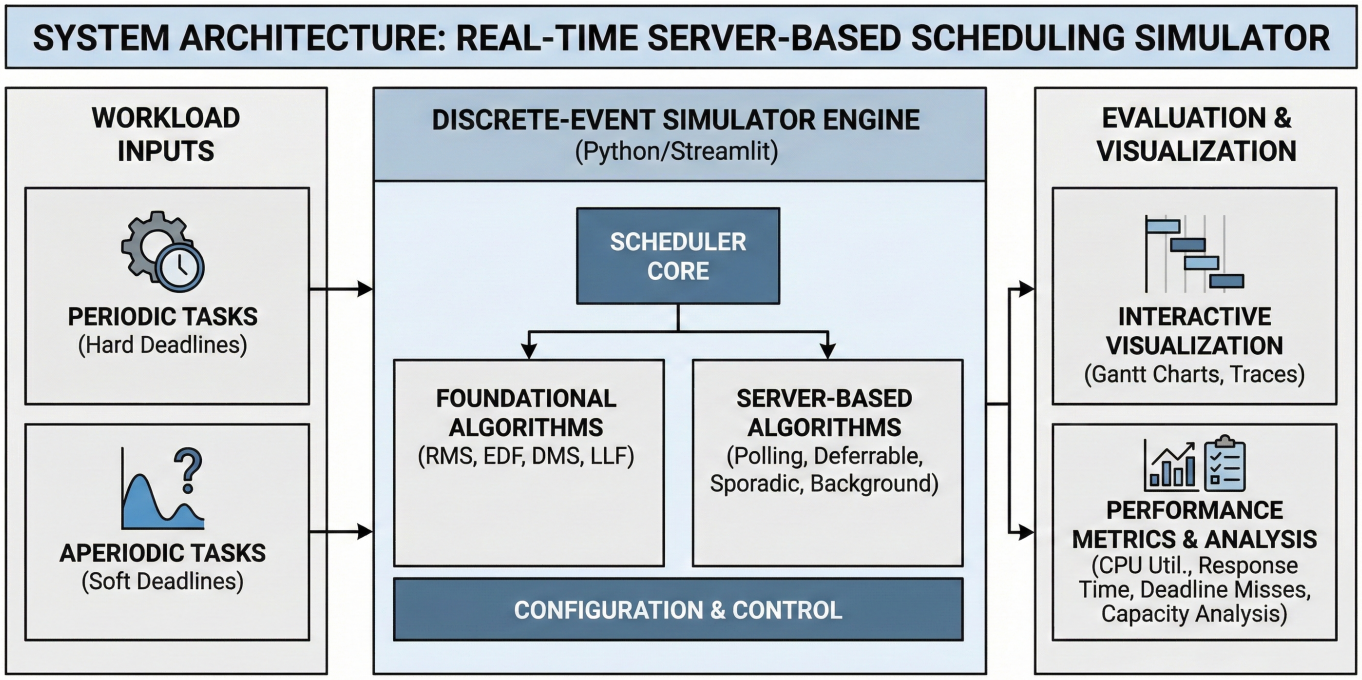

Figure 1: Overview of the simulator's architecture, from mixed workload ingestion to server-based scheduling and analytical output.

Fig. 1. Visual Abstract

1. Introduction

Real-time systems must satisfy timing constraints in addition to functional correctness. When designing mixed periodic-aperiodic workloads, engineers face a critical decision: selecting the appropriate server-based scheduling algorithm and configuring its parameters (capacity C_s and period P_s). While theoretical schedulability analysis provides utilization bounds, it does not reveal the temporal behavior of schedules–when tasks actually execute, how server capacity is consumed and replenished, and how aperiodic response times vary with different configurations.

Real-World Example – Automotive Engine Control Unit (ECU): Consider an automotive ECU that must handle both predictable and unpredictable workloads:

- **Periodic tasks:** Engine control (10ms period), ABS monitoring (5ms period), fuel injection (20ms period)
- **Aperiodic tasks:** Driver button presses, diagnostic requests, error handling

The challenge is: *How can we guarantee that BOTH periodic AND aperiodic tasks meet their deadlines?* This is exactly the problem that server-based scheduling algorithms address–dedicating CPU capacity to aperiodic tasks while protecting periodic task deadlines.

The Gap: Manual schedule construction is tedious and error-prone. Testing algorithms on actual RTOS hardware requires significant development effort and may not be feasible during early design phases. Engineers need a way to explore algorithm behavior before committing to implementation decisions.

Practical Use Cases for Simulation:

1. **Pre-implementation design decisions**: Test different C_s/P_s combinations to find configurations that meet aperiodic response time requirements without compromising periodic schedulability
2. **What-if analysis**: Explore the impact of adding new tasks, changing computation times, or modifying deadlines before deploying changes to production systems
3. **Algorithm selection justification**: Generate Gantt chart visualizations and performance metrics to support design documentation and stakeholder reviews
4. **Workload characterization**: Identify utilization bottlenecks and determine the maximum aperiodic load the system can handle while maintaining deadline guarantees

This project addresses these needs by implementing a discrete-event simulator for four server-based scheduling algorithms from the real-time systems literature:

1. **Polling Server** [2] - Checks for aperiodic tasks periodically; unused capacity is lost
2. **Deferrable Server** [2] - Preserves unused capacity within the same period
3. **Sporadic Server** [1] - Dynamic replenishment provides best aperiodic response time
4. **Background Scheduler** - Baseline that serves aperiodic tasks only during CPU idle time

The simulator provides interactive visualization, allowing users to observe capacity management behavior directly and compare algorithm performance under identical conditions.

---

2. Project Objectives & Scope

2.1. System Model

The system model considers a uniprocessor real-time system with:

- **Periodic tasks**: Tasks τ_i with parameters (C_i, P_i, D_i) representing computation time, period, and deadline
- **Aperiodic tasks**: Tasks with parameters (r_i, C_i, d_i) representing arrival time, computation time, and deadline
- **Server**: A virtual periodic task with capacity C_s and period P_s that services aperiodic requests

This model appears in embedded systems, RTOS environments, and any application requiring deterministic timing guarantees while handling event-driven workloads.

Figure 1: System Model

2.2. Problem Statement

Engineers designing mixed periodic-aperiodic systems must choose between server algorithms and configure parameters, but theoretical formulas alone do not show temporal behavior, and testing on actual RTOS requires significant development effort.

The core problem is: **How do different server capacity management policies affect aperiodic task response times while maintaining periodic task schedulability?**

Research Questions:

- **RQ1:** Can a discrete-event simulator faithfully implement server capacity management policies as described in the literature?
- **RQ2:** Does interactive visualization help users understand the differences between server algorithms?
- **RQ3:** Can parameter exploration (varying C_s/P_s) reveal optimal configurations for specific workloads?

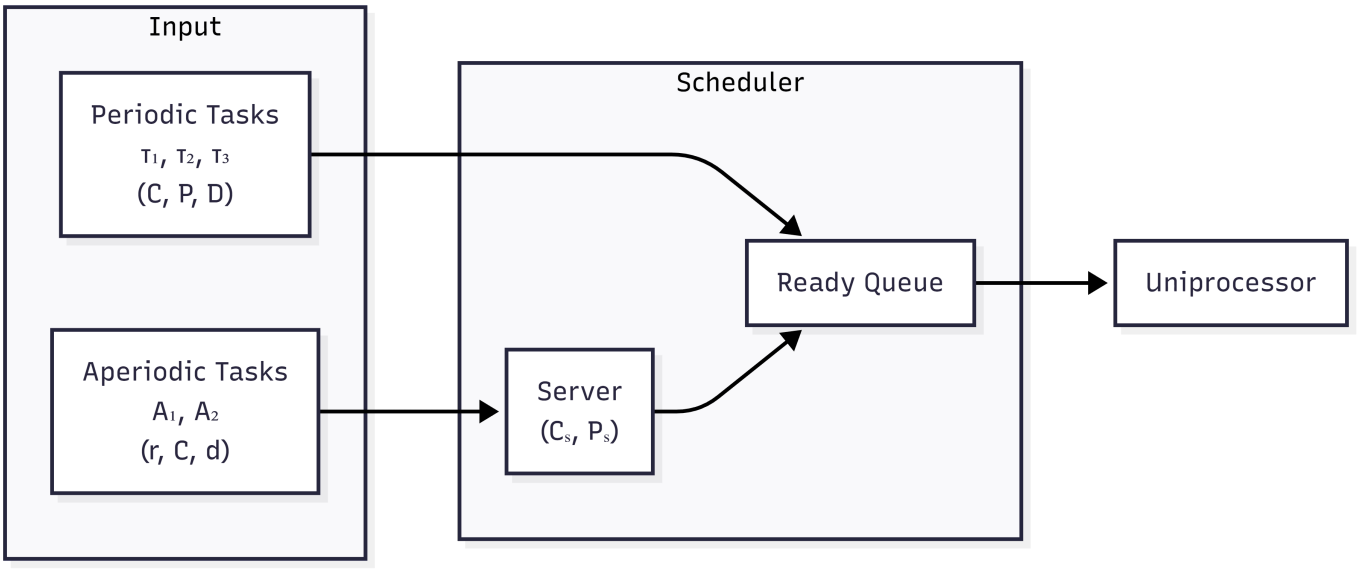

Fig. 2. System Model

2.3. Objectives and Scope

Primary Objectives: 1. Implement four server-based scheduling algorithms with correct capacity management 2. Develop an interactive visualization tool for schedule analysis 3. Compare algorithm behavior through deterministic test scenarios 4. Provide educational demonstrations of server scheduling concepts

Scope: - Uniprocessor scheduling (no multiprocessor considerations) - Discrete-time simulation with unit time steps - Focus on server algorithms under RMS priority assignment - Interactive web-based UI for exploration and analysis

---

3. Solution Methodology / Approach

Our approach is to build a discrete-event simulator that implements server-based algorithms exactly as described in the seminal literature [1][2][3], allowing rapid comparison of algorithm behavior under identical workload conditions. The simulator provides interactive Gantt chart visualization so users can directly observe capacity management behavior–when capacity is consumed, when it is replenished, and how these dynamics affect aperiodic response times.

3.1. Algorithms / Protocols / Architectures

3.1.1. Rate Monotonic Scheduling (RMS): The foundation for server-based scheduling, as proven by Liu and Layland [4]. Tasks are assigned static priorities based on period: shorter period = higher priority. The utilization bound for n tasks is:

$$U = \sum_{i=1}^{n} \frac{C_i}{P_i} \leq n(2^{1/n} - 1)$$

3.1.2. Polling Server: Introduced by Strosnider et al. [2], the Polling Server provides a simple mechanism for aperiodic task handling.

Mechanism: The server is a periodic task with capacity C_s and period P_s. At each server invocation: - If aperiodic tasks are waiting: serve them using available capacity - If no aperiodic tasks: **capacity is lost**

Characteristics: - Simple implementation - Non-bandwidth-preserving - Worst aperiodic response time among server algorithms - Maintains RMS schedulability analysis

Implementation: (`scheduler/core/algorithms/combined.py`, class `PollingServerScheduler`)

```
def _execute_server_slot(self, t: int) -> bool:
    if self.aperiodic_queue and self.server_remaining > 0:
        # Service aperiodic task
```

```
        apt = self.aperiodic_queue[0]
        work_done = min(1.0, self.server_remaining, self.aperiodic_remaining[apt.id])
        self.server_remaining -= work_done
        self.aperiodic_remaining[apt.id] -= work_done
        return True
    else:
        # NO APERIODIC TASKS -> CAPACITY LOST
        self.server_remaining = 0
        return False
```

3.1.3. Deferrable Server: Introduced by Strosnider et al. [2], the Deferrable Server improves upon Polling by preserving unused capacity.

Mechanism: Unlike Polling, the server preserves its capacity when no aperiodic tasks are available. Capacity can be used at any time during the period.

Characteristics: - Bandwidth-preserving - Better aperiodic response time than Polling - Capacity replenished at period boundaries - May interfere with lower-priority periodic tasks

Key difference from Polling:

```
def _execute_server_slot(self, t: int) -> bool:
    if self.aperiodic_queue and self.server_remaining > 0:
        # Service aperiodic task (same as Polling)
        # ...
        return True
    else:
        # CAPACITY PRESERVED (Deferrable behavior)
        # Server defers - does NOT run, keeps capacity for later
        return False  # Let periodic tasks run
```

3.1.4. Sporadic Server: Proposed by Sprunt, Sha, and Lehoczky [1], the Sporadic Server provides optimal aperiodic responsiveness while maintaining RMS schedulability guarantees.

Mechanism: Capacity consumed at time t is replenished at time $t + P_s$. This dynamic replenishment provides the best aperiodic response times.

Characteristics: - Bandwidth-preserving - Best response time among server algorithms - Maintains RMS utilization bound - Most complex implementation (requires replenishment queue)

Replenishment Logic:

```
def _consume_capacity(self, amount: float, t: int) -> None:
    self.server_remaining -= amount
    # Schedule replenishment at t + Ps
    replenish_time = t + self.server_period
    self.replenishment_queue.append((replenish_time, amount))
```

3.1.5. Background Scheduler: **Mechanism:** Aperiodic tasks execute only when the CPU is idle (no periodic tasks ready). This represents the worst-case baseline for aperiodic service.

Characteristics: - Simplest implementation - No interference with periodic tasks - Worst aperiodic response times - Useful as a baseline for comparison

3.1.6. Server Algorithm Comparison Summary: The key difference between server algorithms is how they handle unused capacity:

Algorithm	When No Aperiodic Tasks	Capacity Behavior
Polling	Capacity lost	Non-preserving
Deferrable	Capacity kept until period end	Preserving
Sporadic	Capacity kept, replenish at $t + P_s$	Preserving + Dynamic
Background	N/A (no server concept)	Runs during idle only

Figure 2: Visual comparison of server capacity management strategies

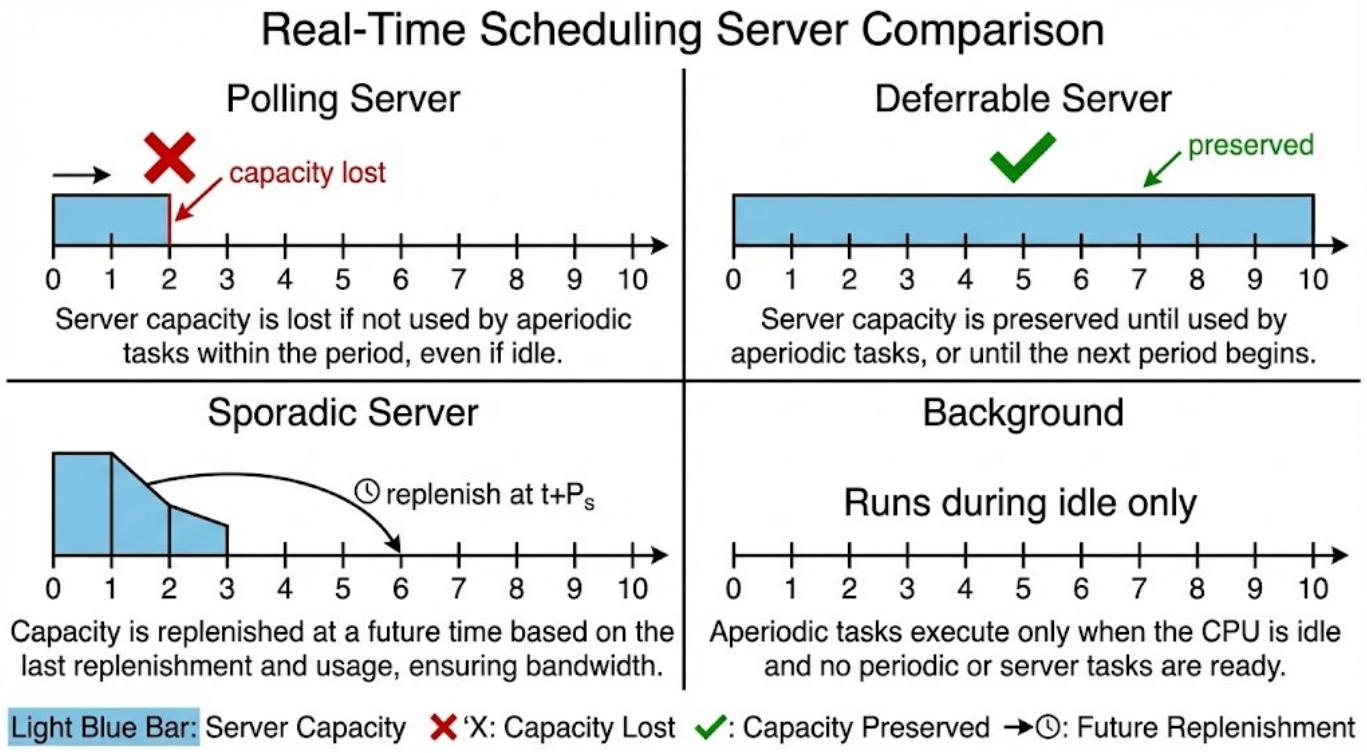

Fig. 3. Server Algorithm Comparison

3.2. Illustrative Example

Consider the following workload:

Periodic Tasks: - P1: C=1, P=10, D=10 (10% utilization) - P2: C=1, P=15, D=15 (6.7% utilization)

Aperiodic Tasks: - A1: arrives at t=0, C=8, deadline=50

Server: C_s=2, P_s=5

Expected Behavior:

Server Type	A1 Completion Time	Explanation
Polling (C_s=2)	~t=17	Needs 4 replenishments, capacity lost when A1 not ready
Polling (C_s=4)	~t=9	Needs 2 replenishments
Polling (C_s=8)	~t=8	Single replenishment sufficient
Deferrable	Faster than Polling	Capacity preserved for immediate use
Sporadic	Fastest	Dynamic replenishment optimizes response
Background	Slowest	Must wait for CPU idle time

This demonstrates how server capacity directly impacts aperiodic task completion time.

---

4. Implementation/Simulation Architecture

4.1. Software Architecture

The simulator follows a layered architecture with clear separation of concerns:

Figure 3: Layered Architecture

Directory Structure:

```
scheduler/
  core/
      task.py              # Task data models
      scheduler_base.py    # Abstract base class
```

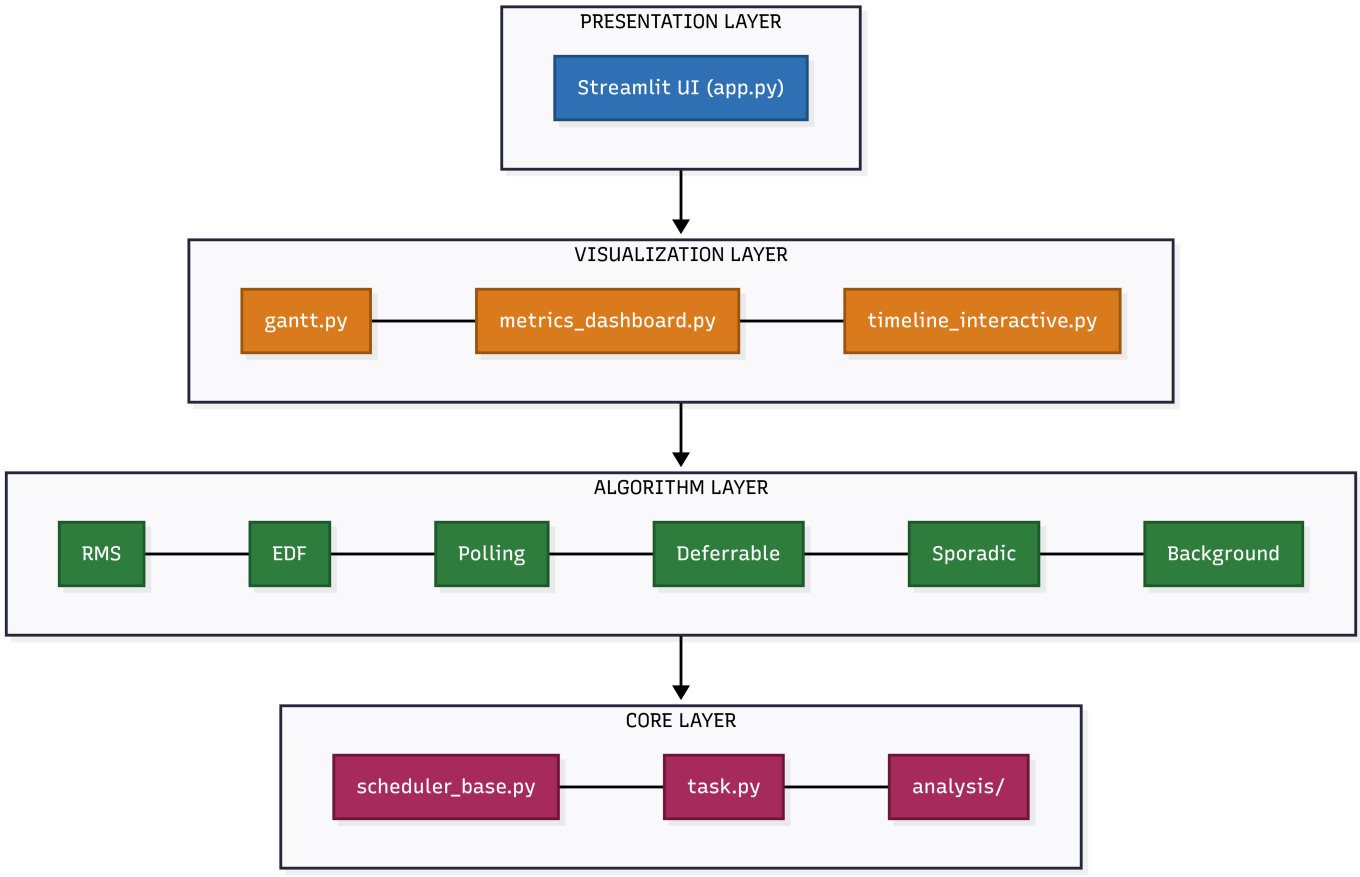

Fig. 4. Layered Architecture

Server schedulers extend this with custom `simulate()` that handles capacity management:

```
class ServerScheduler(SchedulerBase):
    @abstractmethod
    def _handle_replenishment(self, t: int) -> None:
        """Server-specific replenishment policy"""

    @abstractmethod
    def _execute_server_slot(self, t: int) -> bool:
        """Server-specific capacity usage policy"""
```

Figure 4: Class Hierarchy (Template Method Pattern)

4.3. Technology Stack

Component	Technology	Purpose
Core Logic	Python 3.10+	Simulation engine, data models
Web UI	Streamlit	Interactive dashboard
Visualization	Plotly	Interactive Gantt charts, metrics
Data Handling	Pandas	Task tables, data export
Analysis	NumPy	Numerical computations

Layer Responsibilities:

- **Core Logic (Python 3.10+)**: Pure simulation engine with no UI dependencies. Implements all scheduling algorithms, produces `ScheduleResult` objects, and can be tested independently of the visualization layer.

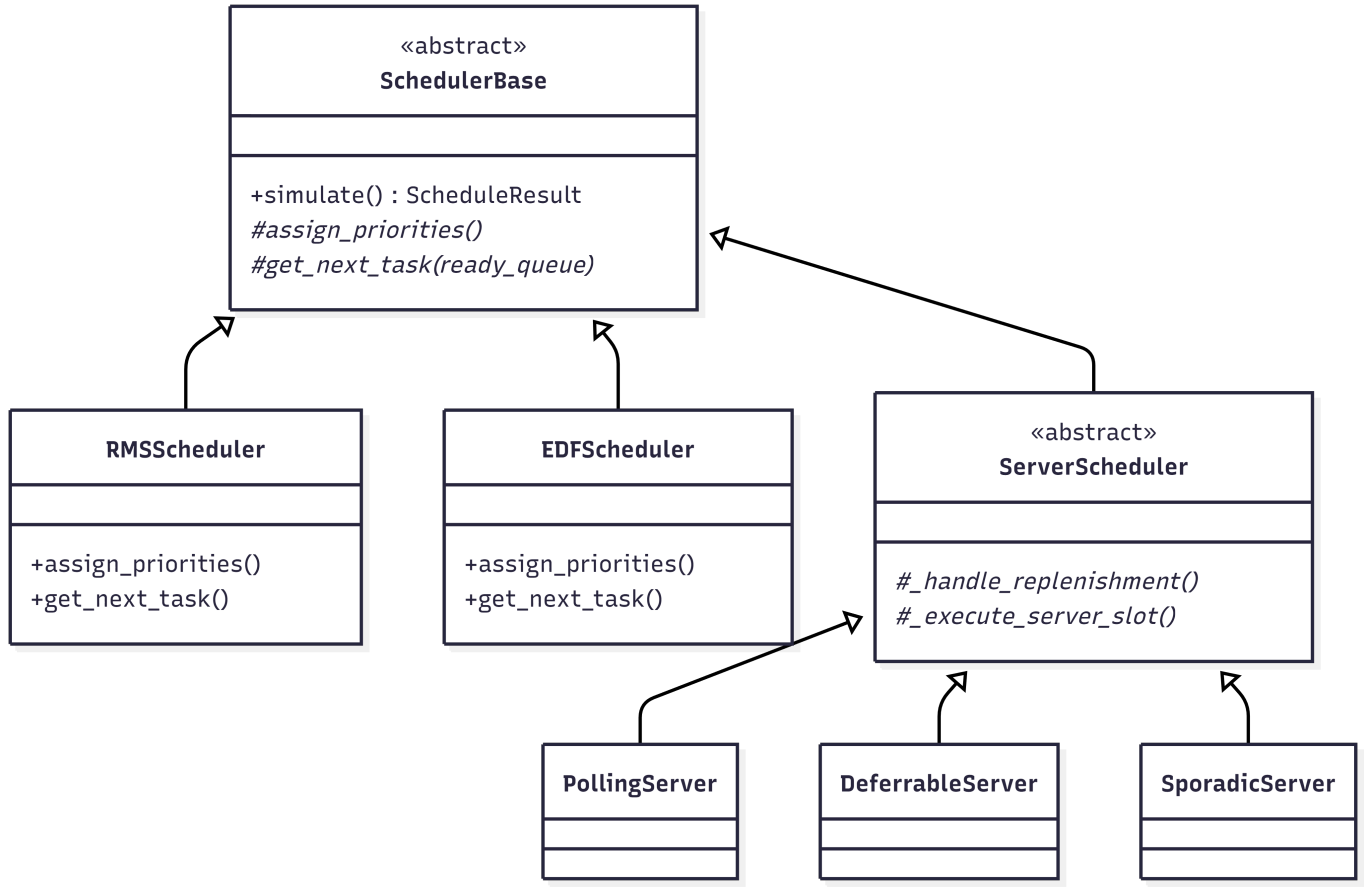

Fig. 5. Class Hierarchy

- **Visualization (Plotly)**: Creates interactive Gantt charts, priority timelines, laxity timelines, and metrics dashboards from `ScheduleResult` data. Completely decoupled from the simulation engine.
- **Web UI (Streamlit)**: Provides the interactive dashboard with algorithm selection, task configuration panels, parameter sliders, and result displays. Orchestrates the core and visualization layers.
- **Data Handling (Pandas)**: Manages task configuration tables, enables CSV/JSON export of results, and transforms simulation data for visualization.

4.4. Key Data Structures

TaskInstance - Represents a single instance of a periodic task:

```
@dataclass
class TaskInstance:
    task_id: str
    instance_number: int
    arrival_time: float
    deadline: float
    remaining_time: float
    completed: bool = False
```

ScheduleEvent - Records simulation events:

```
@dataclass
class ScheduleEvent:
    time: float
    task_id: str
    event_type: str  # 'start', 'complete', 'preempt', 'deadline_miss', 'replenish'
    details: Optional[Dict] = None
```

ScheduleResult - Complete simulation output:

```
@dataclass
class ScheduleResult:
    events: List[ScheduleEvent]
    deadline_misses: List[ScheduleEvent]
    cpu_utilization: float
    context_switches: int
```

4.5. UI Features

The Streamlit-based UI provides:

1. **Algorithm Selection**: Category-based organization (Basic, Server-Based, Precedence, Overload, Aperiodic)
2. **Task Configuration**: Editable data grid for task parameters
3. **Server Configuration**: Capacity (C_s) and Period (P_s) sliders
4. **Preset Library**: 21 curated test scenarios from literature examples
5. **Schedulability Analysis**: Real-time utilization tests with pass/fail indicators
6. **Interactive Gantt Chart**:
   - Task execution intervals with color-coded bars
   - Deadline markers (red triangles) showing deadline boundaries
   - Arrival markers (green circles) indicating task arrival times
   - Server capacity events (`replenish`, `deferred`, `capacity_lost`)
   - Blocked intervals with hatched pattern (when resource protocols enabled)
   - Hover tooltips with detailed event information (task ID, time, remaining computation)
   - Zoom and pan for exploring long simulations
   - Download as PNG for documentation
7. **Metrics Dashboard**: CPU utilization over time, utilization by task (pie chart), event distribution, context switch visualization
8. **Timeline Step Viewer**: Interactive slider to step through simulation events with explanatory text
9. **Priority Timeline**: Dynamic priority visualization for EDF/LLF algorithms showing priority changes over time

---

5. Evaluation

5.1. Validation Approach

Since this is a simulation study (Type 3), evaluation focuses on validating that the simulator correctly implements the algorithms rather than measuring real-world performance:

1. **Correctness**: Do the algorithms produce the expected capacity management behavior as described in the literature?
2. **Observability**: Can users clearly see the differences between server algorithms through visualization?
3. **Parameter Sensitivity**: Does varying C_s/P_s produce the expected effects on aperiodic response time?

Test Environment: - Windows 11, Python 3.11 - Streamlit 1.28+, Plotly 5.18+ - Discrete-event simulation with unit time steps

5.2. Correctness Validation

The following behaviors were verified against the algorithm descriptions in [1][2][3]:

Server Type	Expected Behavior	Verified?
Polling Server	Capacity lost when no aperiodic tasks pending	Yes (`capacity_lost` events observed)
Deferrable Server	Capacity preserved within period	Yes (`deferred` events observed)
Sporadic Server	Replenishment scheduled at $t + P_s$	Yes (`replenish` events at correct times)

Server Type	Expected Behavior	Verified?
Background	Aperiodic runs only during CPU idle	Yes (no preemption of periodic tasks)

Performance Metrics Collected:

Metric	Definition
CPU Utilization	(Busy time / Duration) × 100%
Context Switches	Count of task execution start events
Deadline Misses	Count of tasks completing after deadline
Aperiodic Response Time	Completion time - Arrival time

5.3. Parameter Sensitivity Results

5.3.1. Server Capacity Effect (RQ3): **Workload:** P1(C=1,P=10), P2(C=1,P=15), A1(C=8, arrives t=0)

C_s	P_s	A1 Response Time	Server Replenishments
2	5	17	4
4	5	9	2
8	5	8	1

Finding: Server capacity directly impacts aperiodic response time. Larger C_s reduces the number of replenishment cycles needed, demonstrating that parameter exploration (RQ3) reveals optimal configurations.

5.3.2. Server Algorithm Comparison (RQ1, RQ2): **Workload:** Basic Server Example (SERVER_EXAMPLE_1) - Periodic: P1(C=2,P=10), P2(C=1,P=8) - Aperiodic: A1(t=3,C=2), A2(t=8,C=1), A3(t=15,C=2) - Server: C_s=2, P_s=5

Algorithm	Avg Response Time	Observable Behavior
Polling	Moderate	Gantt shows `capacity_lost` events when A1 not ready
Deferrable	Good	Gantt shows capacity preserved via `deferred` events
Sporadic	Best	Gantt shows dynamic `replenish` events
Background	Worst	Aperiodic task runs only in gaps between periodic tasks

Finding: Interactive visualization clearly shows the behavioral differences between server types (RQ2), and the simulator faithfully implements the capacity management policies (RQ1).

Gantt Chart Visualization Features Supporting RQ2:

- Color-coded task bars distinguish periodic tasks from aperiodic tasks and server activity
- Red triangle markers indicate deadline boundaries, making deadline misses immediately visible
- Green circle markers show task arrival times for aperiodic tasks
- Hover tooltips reveal detailed event information (remaining computation time, server capacity)
- Zoom/pan capabilities allow exploration of specific time intervals
- Server events (`replenish`, `deferred`, `capacity_lost`) appear as labeled markers on the timeline

5.3.3. Schedulability Analysis Validation: **RMS Utilization Test:** - Task set: T1(C=2,P=4), T2(C=1,P=8) - U = 2/4 + 1/8 = 0.625 - Bound (n=2): 2(2^0.5 - 1) = 0.828 - Result: **SCHEDULABLE** (0.625 <= 0.828)

EDF Utilization Test: - Task set: T1(C=1,P=3), T2(C=4,P=6) - U = 1/3 + 4/6 = 1.0 - Result: **SCHEDULABLE** (U <= 1.0)

5.4. Reproducibility

To reproduce these results:

1. **Start the simulator**: `streamlit run scheduler/app.py`
2. **Load preset**: Select "Server Capacity Demo" from the preset dropdown
3. **Run simulation**: Click "Run Simulation" button
4. **Vary parameters**: Adjust C_s slider to 2, 4, or 8 and re-run
5. **Observe Gantt chart**: Note the aperiodic task completion time changes

All presets and configurations are stored in `scheduler/configs.py` for reproducibility.

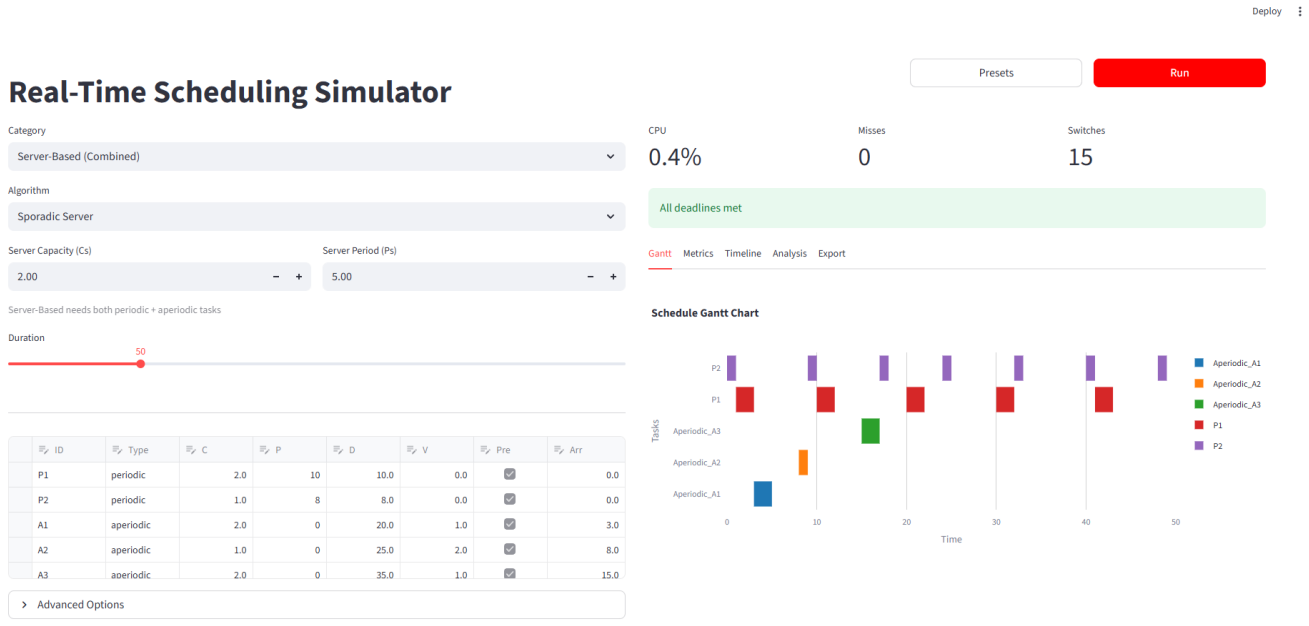

Fig. 6. Figure 5: Sporadic Server Gantt Chart - showing task execution (colored bars), deadline markers (red triangles), and server replenishment events

6. Conclusions

6.1. Contributions

This project delivers the following contributions:

1. **Open-source simulator**: Python/Streamlit implementation of four server-based scheduling algorithms (Polling, Deferrable, Sporadic, Background) with correct capacity management behavior. Source code available at: https://github.com/shahabafshar/Scheduler
2. **Interactive visualization**: Plotly-based Gantt charts that display capacity events (`replenish`, `deferred`, `capacity_lost`) so users can observe algorithm differences directly
3. **Preset library**: 21 curated task sets from literature examples for experimentation and validation
4. **Parameter exploration**: Users can modify C_s/P_s via sliders and immediately observe the effect on aperiodic response times
5. **Schedulability analysis**: Built-in utilization tests for RMS, EDF, and DMS with pass/fail indicators

6.2. Research Question Answers

RQ1: Can a discrete-event simulator faithfully implement server capacity management policies?

Answer: YES. The simulator correctly implements capacity management for all four server algorithms, verified by observing expected event patterns in the simulation timeline:

- Polling Server generates `capacity_lost` events when no aperiodic tasks are pending
- Deferrable Server preserves capacity via `deferred` events (no `capacity_lost` events)
- Sporadic Server schedules `replenish` events at exactly $t + P_s$ after capacity consumption
- Background Scheduler executes aperiodic tasks only during CPU idle intervals

This behavioral verification confirms faithful implementation of the algorithms as described in the literature (Sprunt et al., Strosnider et al.).

RQ2: Does interactive visualization help users understand algorithm differences?

Answer: YES. The Gantt chart visualization provides clear visual differentiation between server algorithms:

- Color-coded task bars distinguish periodic (blue) from aperiodic (orange) tasks
- Server capacity events (`replenish`, `deferred`, `capacity_lost`) are labeled directly on the timeline
- Users can visually compare how the same workload behaves under different server algorithms by switching between them
- Hover tooltips provide detailed event information (remaining computation time, server capacity)
- Zoom and pan capabilities enable exploration of specific time intervals

Users can immediately see when capacity is lost (Polling) versus preserved (Deferrable) versus dynamically replenished (Sporadic).

RQ3: Can parameter exploration reveal optimal configurations?

Answer: YES. Parameter exploration via sliders enables rapid discovery of optimal server configurations:

- Varying C_s from 2 to 8 with the Server Capacity Demo preset demonstrates response time reduction from 17 to 8 time units (53% improvement)
- Increasing server capacity reduces the number of replenishment cycles needed (4 to 2 to 1)
- This insight cannot be obtained from utilization formulas alone–simulation reveals the temporal behavior

Users can find configurations that meet response time requirements without trial-and-error deployment on real systems.

6.3. Recommendations for Future Work

1. **Multi-core Extension**: Implement partitioned and global scheduling algorithms for multiprocessor systems, including task migration policies and load balancing strategies.
2. **Resource Protocol Integration**: Complete integration of Priority Inheritance Protocol (PIP) and Priority Ceiling Protocol (PCP) into the main simulation loop for shared resource scenarios.
3. **RTOS Trace Import**: Enable comparison of simulation results with actual execution traces from FreeRTOS, Zephyr, or other RTOS platforms to validate simulation fidelity.
4. **Formal Verification**: Apply model checking techniques to provide formal schedulability guarantees beyond utilization-based tests.
5. **Total Bandwidth Server (TBS)**: Implement EDF-based server with dynamic deadline assignment [3] for improved aperiodic responsiveness under EDF scheduling.
6. **Statistical Analysis**: Implement random task generation using UUniFast algorithm for systematic exploration of algorithm behavior across diverse workload scenarios.
7. **Multi-Server Configurations**: Support multiple servers with different priorities and capacities for more flexible aperiodic task handling.

6.4. Limitations

1. **Single-processor only**: The simulator currently supports uniprocessor scheduling. Multi-core scenarios with task migration, partitioned scheduling, and global scheduling are not yet implemented.
2. **No resource contention modeling**: Priority Inheritance Protocol (PIP) and Priority Ceiling Protocol (PCP) are implemented in the codebase (`scheduler/core/protocols/`) but not yet integrated into the main simulation loop. Shared resource scenarios cannot be simulated.
3. **Limited RQ2 validation**: The assessment of visualization effectiveness (RQ2) is based on feature implementation and informal observation. A formal user study with quantitative metrics would provide stronger evidence of visualization effectiveness.

4. **Simplified execution model**: The simulator does not model I/O delays, cache effects, interrupt latency, or context switch overhead beyond basic counting. Results represent idealized discrete-time behavior.
5. **Deterministic workloads only**: Random/stochastic task generation using algorithms like UUniFast is not implemented. All task sets must be manually defined or loaded from presets.
6. **No hardware validation**: Results are simulation-based and have not been validated against actual RTOS implementations (e.g., FreeRTOS, Zephyr). Simulation fidelity to real hardware behavior is not verified.

Self-Assessment of Project Completion

Project Learning Objectives	Status	Pointers
Self-contained description of project goal, scope, and requirements	Fully Completed	Section 2, pages 2-3
Self-contained description of solutions (algorithms/protocols)	Fully Completed	Section 3, pages 3-6
Adequate description of implementation details	Fully Completed	Section 4, pages 6-8
Testing and evaluation - test cases, metrics, results	Fully Completed	Section 5, pages 8-10
Overall Project Success Assessment	**Fully Successful**	

---

7. References

[1] B. Sprunt, L. Sha, and J. Lehoczky, "Aperiodic task scheduling for Hard-Real-Time systems," *Real-Time Syst*, vol. 1, no. 1, pp. 27-60, June 1989, doi: 10.1007/BF02341920.

[2] J. K. Strosnider, J. P. Lehoczky, and L. Sha, "The deferrable server algorithm for enhanced aperiodic responsiveness in hard real-time environments," *IEEE Transactions on Computers*, vol. 44, no. 1, pp. 73-91, 1995, doi: 10.1109/12.368008.

[3] M. Spuri and G. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179-210, Mar. 1996, doi: 10.1007/BF00360340.

[4] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *J. ACM*, vol. 20, no. 1, pp. 46-61, Jan. 1973, doi: 10.1145/321738.321743.

Prepared by: Shahab Afshar **Date:** December 2025 **Instructor:** Dr. G. Manimaran **Department:** Electrical and Computer Engineering, Iowa State University